PERSONAL EXPENSE TRACKER

VITyarthi Build Your Own Project

A simple command-line Python application for expense management, validation, local persistence, and reporting.

Student Name: _____

Course / Subject: _____

GitHub Repository: https://github.com/YOUR_USERNAME/YOUR_REPO

1. Introduction

Personal Expense Tracker is a command-line application developed in Python. It allows a user to maintain a local record of personal expenses and view simple summaries. The design focuses on a small scope, standard-library-only implementation, modular source files, validation, persistence, and automated tests.

2. Problem Statement

People may record daily spending in scattered notes or may not keep a consistent record. The project addresses this by providing one small terminal application for creating, viewing, editing, deleting, and summarizing expense records.

3. Functional Requirements

The system provides three major functional areas: (1) expense CRUD management, (2) validation and JSON data persistence, and (3) reporting and analytics. The command-line workflow gives clear inputs and outputs for each operation.

4. Non-Functional Requirements

Usability: the application uses a numbered menu and readable messages. Reliability: data is stored persistently in JSON and writes use a temporary file before replacement. Maintainability: presentation, business logic, validation, reporting, models, and storage are separated. Resource efficiency: the project uses a small local JSON file and only Python standard-library modules. Error handling: invalid input and storage errors are converted into readable messages.

5. System Architecture

The application uses a lightweight layered architecture. The CLI receives user input, ExpenseManager performs business operations, validators check values, reports calculate summaries, and ExpenseStorage reads/writes the JSON file. Expense is the domain model.

6. Design Diagrams

The repository contains separate architecture, workflow, UML, and test-case documents under docs/. A condensed workflow is shown below.

7. Design Decisions & Rationale

Python was selected because the project is intentionally small and should run without external services. JSON was selected instead of a database to minimize setup and dependencies. Dataclasses provide a clear domain model. The manager class centralizes business rules, while validation functions keep input checks reusable. unittest is used because it is included with Python.

8. Implementation Details

The main entry point is main.py. The expense_tracker package contains models.py, validators.py, storage.py, expense_manager.py, reports.py, and cli.py. The data directory contains the JSON store. The tests directory contains unit tests for persistence, CRUD behavior, reporting, and validation.

9. Testing Approach

Testing uses Python's built-in unittest framework. Tests cover adding and reloading data, updating and deleting records, report calculations, and rejection of an invalid amount. Additional manual test cases are documented in docs/test_cases.md.

10. Screenshots / Results

The project is a terminal application. Run python main.py to see the menu. Use option 1 to add records, option 2 to list them, and option 5 to see total, category, and monthly summaries. The README contains the exact execution commands.

11. Challenges Faced

The main implementation challenges are keeping file storage reliable, separating business logic from user interaction, and handling invalid input without terminating the program. These are addressed through a storage class, manager class, validation functions, and exception handling in the CLI.

12. Learnings & Key Takeaways

The project demonstrates Python modules and packages, dataclasses, file handling, JSON serialization, input validation, CRUD design, separation of concerns, automated testing, and basic software documentation.

13. Future Enhancements

Possible future enhancements include CSV export, configurable budgets, search/filter commands, password-protected local data, a database backend, charts, and a web interface. These are outside the current minimal scope.

14. References

1. Python Standard Library documentation: https://docs.python.org/3/library/

2. Python unittest documentation: https://docs.python.org/3/library/unittest.html

3. VITyarthi Build Your Own Project instructions supplied with this submission.

Condensed Workflow

Start	Load JSON	Show Menu	Choose Operation	Validate/Process	Save/Display	Return/Exit

Use Case Summary

Actor	Use Case	Outcome
User	Add Expense	New validated record is stored
User	View Expenses	Saved records are displayed
User	Update Expense	Existing record is changed
User	Delete Expense	Existing record is removed
User	View Reports	Totals and summaries are displayed